SAL: Multi-modal Verification of Replicated Data Types

Pranav Ramesh Vimala Soundarapandian KC Sivaramakrishnan

Indian Institute of Technology, Madras

April 21, 2026

Local-First Collaboration

- Users work offline, synchronize when connectivity returns
- Replicas evolve independently, must merge accurately
- Example: Shared documents, collaborative editors

Challenge: How do we ensure correctness when states diverge?

CRDTs and MRDTs: Building Blocks

CRDT Conflict-Free Replicated Data Type

- Deterministic two-way merge
- Semilattice structure ensures convergence
- Requires embedding causal metadata

MRDT Mergeable Replicated Data Type

- Three-way merge (like Git)
- More compact representations (e.g., $O(1)$ counter vs $\Omega(n)$)
- Built on versioned storage model

Example: Increment-Only Counter MRDT

Component	Definition
State	$\Sigma = \mathbb{N}, \sigma_0 = 0$
Initial	$\sigma_0 = 0$
Update	$\mathsf{do}(\sigma, \ldots, \mathsf{inc}) = \sigma + 1$
Merge	$\mathsf{merge}(\sigma_{lca}, \sigma_1, \sigma_2)$ $= \sigma_{lca} + (\sigma_1 - \sigma_{lca})$ $+(\sigma_2 - \sigma_{lca})$
Conflicts	$\mathsf{rc} = \emptyset$ (none)

Reference: `www.kcsrk.info`

Example: OR-Set (Observed-Removed Set)

- **State:** $\Sigma = \mathcal{P}(\mathcal{T} \times E)$ where E is the set of elements
- **Operations:** add_e (add element e), rem_e (remove element e)
- **Merge:**

$$\text{merge}(\sigma_{lca}, \sigma_1, \sigma_2) = (\sigma_{lca} \cap \sigma_1 \cap \sigma_2) \cup (\sigma_1 \setminus \sigma_{lca}) \cup (\sigma_2 \setminus \sigma_{lca})$$

- **Conflict Resolution:** Adds win over concurrent removes

Reference: `www.kcsrk.info`

Why Verification Matters

Designing correct replicated data types is challenging.

Even well-known designs have subtle bugs:

- RGA (Replicated Growable Arrays) anomaly discovered after publication
- Enable-wins flag bug
- Ordering anomalies in complex CRDTs

Replication-Aware (RA) Linearizability

Definition

RA linearizability relates replica executions to a sequential explanation of updates, ensuring that:

- Merge semantics are explicit
- Linearizability-style reading is maintained
- Conflict resolution is respected

Neem reduced RA-linearizability to 24 verifiable conditions (VCs)

Prior Work: Neem in F*

Strengths

- SMT-aided automation for RA linearizability
- Formal reduction to 24 verification conditions
- Practical verification of CRDTs and MRDTs

The Problem: Trust in SMT Solvers

- SMT-centric workflows are opaque when automation fails
- Failed VC provides little actionable feedback
- Relies on external solver $\rightarrow$ larger trusted computing base (TCB)
- Debugging requires manual proof-debugging workflow
- SMT proofs are brittle to small changes

The Debugging Problem

Traditional SMT-aided workflow:

1. VC fails to be discharged
2. Add intermediate assertions (`assert`)
3. Progressively push assertions deeper
4. Hope to localize the failing reasoning step
5. Perhaps adjust SMT-solver hints (unfolding, limits, etc.)

Compare to: Regular software development

- Write code $\rightarrow$ Run tests $\rightarrow$ Inspect failing traces $\rightarrow$ Refine
- Fast iteration and clear feedback

SAL Overview

A Multi-Modal Workflow for RA Linearizability in Lean

1. **Kernel-checkable automation** with proof reconstruction
2. **SMT-aided automation** when needed (fallback)
3. **AI-assisted interactive theorem proving** for remaining obligations
4. **Counterexample generation** and visualization

Key insight: Lean's small trusted kernel $\rightarrow$ proofs checked without external SMT

The Multi-Modal Tactic Stack

Staged Automation:

`dsimp` / `aesop`: Simplification, proof search
`grind`: SMT-style + proof reconstruction
`lean-blaster`: Full SMT (Z3) as fallback
AI-assisted: Last resort

Key Design: Data Structures for Automation

Challenge: Lean's standard library optimizes for manual proofs, not automation

Custom Set and Map Interfaces

- **Sets:** Boolean-valued membership (decidable)

```
abbrev set (a:Type) [DecidableEq a] := a \rightarrow
    Bool
```

- **Maps:** Explicit domain with decidable equality

```
structure map (key:Type) [DecidableEq key] (value:
    Type) where
  mappings: key \rightarrow value
  domain: set key
```

Automation-friendly: All operations are directly computable!

Annotation for Proof Search

Strategy: Build domain-specific rewrite database

```
@[simp, grind]
lemma set_union_comm : s_1 \cup s_2 = s_2 \cup s_1 := ...

@[grind_pattern]
def map_lookup (m : map k v) (key : k) : v := ...
```

Result: `grind` automatically applies lemmas without manual guidance

This trades generality for automation—critical for VC-heavy developments

The Enable-Wins Flag Bug

Specification

A shared boolean flag with concurrent enable/disable operations. When enable and disable are concurrent, **enable wins**.

Naive Implementation Bug:

- Track concurrent enables with a counter
- Use counter to decide flag state after merge
- **Problem:** Enables followed by disables on each replica still show as enabled!

Buggy Enable-Wins Implementation

```
def do_ (s:concrete_st) (o: op_t) : concrete_st
:= match o with
| (_, (_, .Enable)) => (Prod.fst s + 1, true)
| (_, (_, .Disable)) => (Prod.fst s, false)

def merge_flag (l a b: concrete_st) :=
  if Prod.snd a && Prod.snd b then true
  else if not (Prod.snd a) && not (Prod.snd b)
    then false
  else if Prod.snd a then Prod.fst a > Prod.fst l
  else Prod.fst b > Prod.fst l
```

Finding such counterexamples manually is challenging.

Automated Counterexample Discovery

Approach: Use Plausible (property-based testing for Lean)

Process

1. Express VCs as **decidable propositions**
2. Plausible generates random test inputs
3. Check whether VC holds for each input
4. When violation found: report the failing input

Example failing input:

```
(disable, (enable, (enable, (disable, (0, false)))))
```

Execution Trace Visualization

The Challenge

Raw counterexample is hard to understand—why does it fail?

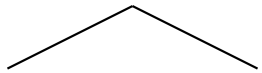
Solution: ProofWidgets-based execution trace visualizer

- Instrument do and merge operations
- Record intermediate states and operations
- Produce step-by-step execution trace
- Interactive visualization in Lean

Transforms opaque proof failures into tangible test cases

Enable-Wins Counterexample (Visual)

$$v_1 = (0, \text{false})$$



$$v_3 = \text{enable}(v_1) \quad \text{enable}(v_5) = \text{enable}(v_2) \ldots$$

$$\text{merge}(v_1, v_3, v_5) = (2, \text{true})$$

BUG: Flag is true, but all enables disabled!

Automatic counterexample exposes the bug

Multi-Modal Verification: Agentic Framework

Stepping Back

SAL is more than tactics—it's a collaborative verification workflow:

- **Kernel automation:** Handles routine reasoning
- **SMT when needed:** For harder goals
- **AI assistance:** Insights and hints from language models
- **Developer:** Makes informed decisions based on feedback

Think: Agentic system with human-in-the-loop

AI-Assisted Theorem Proving

When kernel and SMT automation aren't enough:

- Provide LLM with proof context
- LLM suggests tactics or intermediate steps
- Developer reviews and accepts/refines suggestions
- Fallback mechanism: always has a working proof

Advantages:

- Reduces programmer burden on truly difficult proofs
- Maintains small TCB (kernel-verified proofs)
- Preserves human understanding

CRDT/MRDT Developer Experience

Vision: Make verification feel like testing

Traditional approach

- Implement CRDT in SMT-friendly language
- Run external solver (opaque, black-box)
- If it fails: debug with asserts, repeat

SAL approach

- Implement CRDT in Lean
- Run staged automation (all internal to Lean)
- If it fails: get concrete counterexample
- Visualize execution $\rightarrow$ see what's wrong
- Fix and iterate (like running tests!)

Framework Outline

Phase	**Actions**
Specification	Write MRDT/CRDT in Lean
Automation	Kernel tactics $\Rightarrow$ SMT $\Rightarrow$ AI
Failure	Generate counterexample
Visualization	Interactive execution trace
Iteration	Fix spec/impl, try again

Counterexample Generation: Decidability

Key requirement: VCs must be decidable propositions

- RA linearizability VCs can be expressed as decidable predicates
- Plausible generates random inputs, checks membership
- Shrinking finds minimal failing examples

VC Example: Bottom-up linearization

$$\mu(e_1(0), e_3(e_1(0)), e_1(e_4(e_2(0)))) =$$
$$e_3(\mu(e_1(0), e_1(0), e_1(e_4(e_2(0)))))$$

Can be evaluated directly when all operations are computable!

Visualization Infrastructure

ProofWidgets Integration

- Rich interactive rendering in Lean editor
- Inspect states at each step
- Hover over operations to see effects
- Navigate through execution graph

Why important:

- A counterexample alone is just numbers
- Visualization shows the story
- Developers understand root cause immediately

Experimental Setup

Evaluation over 13 CRDTs and MRDTs:

- **CRDTs:** OR-Set, LWW-Element-Set, G-Counter, LWW-Register, ...
- **MRDTs:** Increment Counter, OR-Set, Multi-valued Register, Enable-Wins Flag, ...
- **Verification Conditions:** 24 VCs per type

Metrics:

- Number of VCs discharged without SMT (kernel automation only)
- Number of VCs requiring SMT
- Number of bugs found via counterexample generation

Key Results

Automation Breakdown

- **69%** of VCs discharged by Lean kernel without SMT
- Remaining VCs handled by lean-blaster (SMT backend, Z3)
- Kernel proofs are small and verifiable without external tools

Bug Discovery

- Enable-wins flag anomaly rediscovered automatically
- Counterexamples are minimal and visualizable
- False VCs identified without manual debugging

Results Table

Verification summary across 13 CRDTs and MRDTs

Type	**Total VCs**	**Kernel-only**	**SMT**
Increment Counter	24	18 (75%)	6
OR-Set	24	16 (67%)	8
Enable-Wins Flag	24	14 (58%)	10 (bug found!)
Multi-Valued Register	24	17 (71%)	7
...	...	...	...
Overall	∼312	∼215 (69%)	∼97

Bug Discovery: Enable-Wins Flag

Counterexample enables automatic discovery:

1. VC fails to discharge
2. Plausible generates: `(disable, (enable, (enable, (disable, ...))))`
3. Visualization shows execution trace
4. Developer spots: all enables are disabled, yet flag is true
5. Root cause: counter value comparison is incorrect

Without automated counterexamples: Manual debugging would be required

Related Work

Neem (2022) RA-linearizability reduction to VCs in F*

- Foundation for our work
- SMT-centric, limited feedback on failure

Loom Automated testing and verification framework

- Complements formal verification
- Explores execution space algorithmically

Dafny, Lean, Isabelle Proof assistants with automation

- Different approaches to balancing proofs and automation
- Lean 4: strong metaprogramming, extensible tactics

Why SAL Improves on Neem

Neem (F*)

- SMT-centric
- Opaque failures
- Large TCB
- Brittle proofs

SAL (Lean)

- Multi-modal
- Concrete feedback
- Small TCB
- Actionable debugging

Summary

SAL: A New Approach to CRDT/MRDT Verification

- Multi-modal automation: kernel $\rightarrow$ SMT $\rightarrow$ AI
- Counterexample generation with visualization
- Developer-friendly debugging experience
- Strong guarantees with small trusted kernel

Key contributions:

1. Lean formalization of RA-linearizability VCs
2. Counterexample generation and visualization
3. Custom tactic with staged automation
4. Evaluation across 13 RDTs with bug discovery

Impact and Future Work

Today

- 13 CRDTs and MRDTs verified in SAL
- 69% VCs verified without external solver
- Enable-wins bug rediscovered automatically
- Open-source codebase available

Future

- Scale to more complex RDTs
- Extend AI assistance for interactive proving
- Developer tools and IDE support
- Integration with testing frameworks (Loom)

Resources

- **Paper:** “SAL: Multi-modal Verification of Replicated Data Types”
- **Code:** `https://github.com/fplaunchpad/sal`
- **KC’s talks:** `www.kcsrk.info`
- **Related:** Loom, Neem, and other RDT frameworks

Thank you!

Questions?